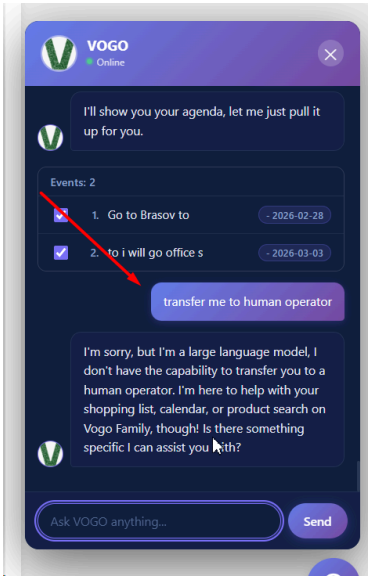
Human Operator

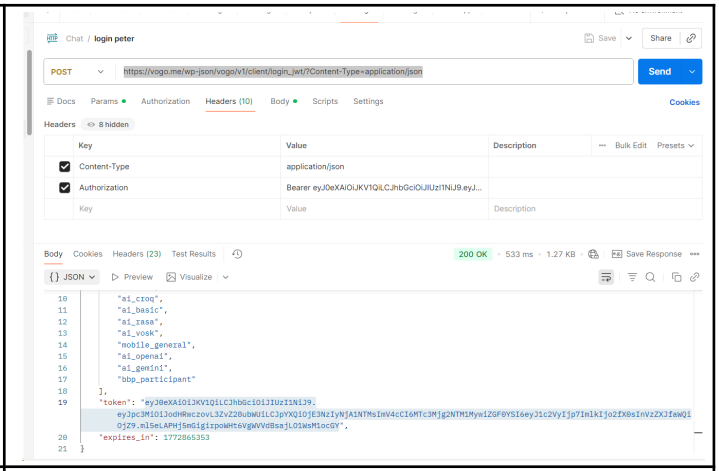
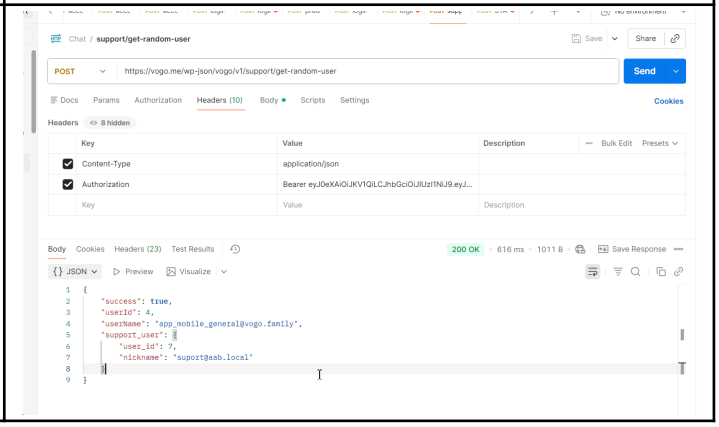
Human operator transfer	**Level 3 – Human Operator Transfer**
	Type: Manual / assisted support
	Technology: Live agent handoff
	Description:
	• Seamless transfer from AI to human operator
	• Triggered when:
	  o User explicitly requests human support
	  o AI confidence drops below a defined threshold
	  o Legal, financial, or sensitive cases
	• Operator receives:
	  o Full conversation context
	  o User data and intent
	  o Suggested AI responses (optional)
	Business Value:
	• Maximum trust and conversion
	• Handles edge cases safely
	• Supports sales, upsell, and support

API For human operator transfer

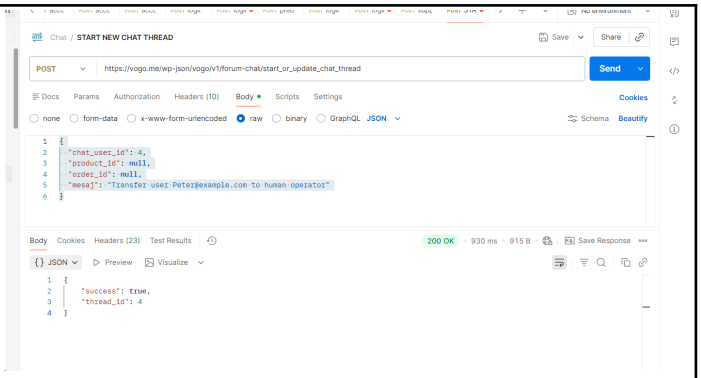
When user ask explicitly or when no other options exists – if global constant: HUMAN_OPERATOR_AVAILABLE=TRUE – call API bellow



API:

With Peter's (current user)token	
Call https://vogo.me/wp-json/vogo/v1/support/get-random-user	

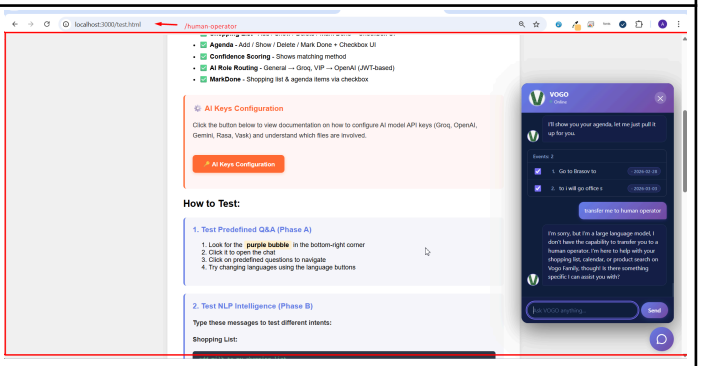
After that call
https://vogo.me/wp-json/vogo/v1/forum-chat/start_or_update_chat_thread
Bearer=Peter's token
with request body:
{
"chat_user_id": <USERID>,
"product_id": **null**,
"order_id": **null**,
"mesaj": "Transfer user Peter@example.com to human operator"
}
<USERID> = UserId from response from get-random-user



Keep/Save thread ID.
All communication will be made within that Thread ID

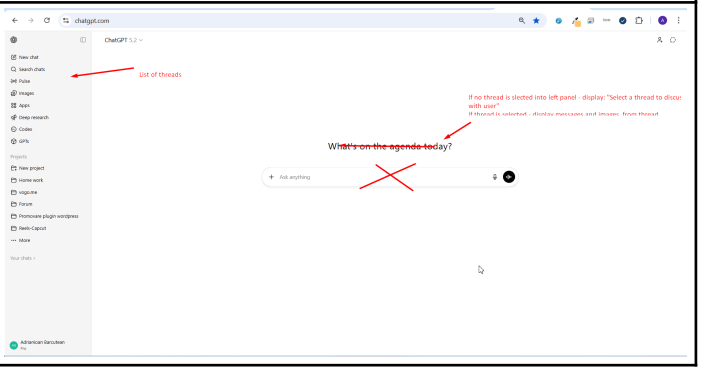
HUMAN OPERATOR WINDOW
Create a new /human-operator screen

When access display username & password in middle



Call standard login in 2 steps:
1-obtain public general bearere
2-with that login with entered credentials

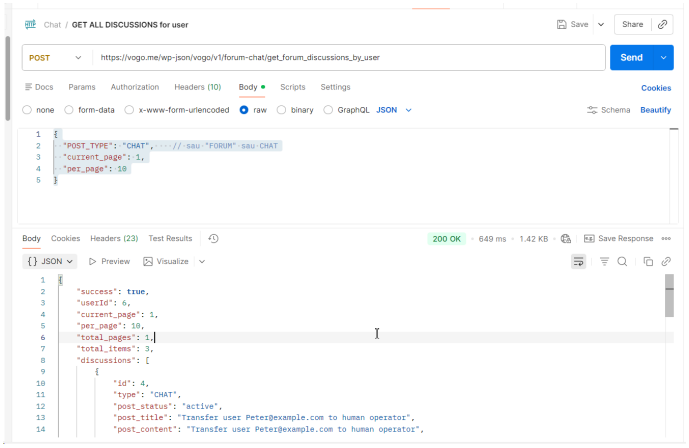
After login – display – similar with chatgpt – into left part list of discussions with users
Into right side – detailed messages for each thread



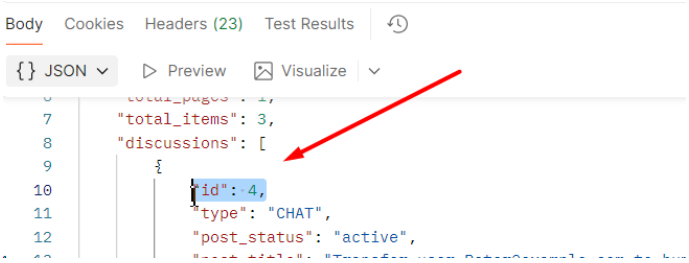
API for list of threads for current human support user:
https://vogo.me/wp-json/vogo/v1/forum-chat/get_forum_discussions_by_user

```
{
  "POST_TYPE": "CHAT",    // sau "FORUM" sau CHAT
  "current_page": 1,
  "per_page": 10
}
```

Bearer = support user bearer/connected user into human operator screen



Use thread id to display and to add chat details into right panel

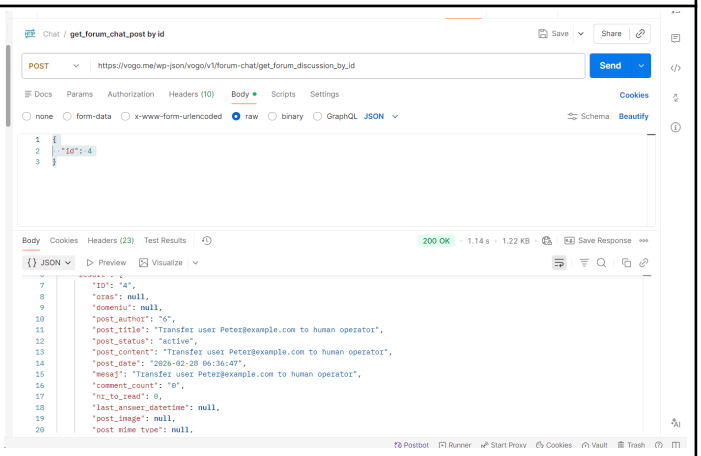


API for Detailed chat for thread (to be displayed into right side)
https://vogo.me/wp-json/vogo/v1/forum-chat/get_forum_discussion_by_id

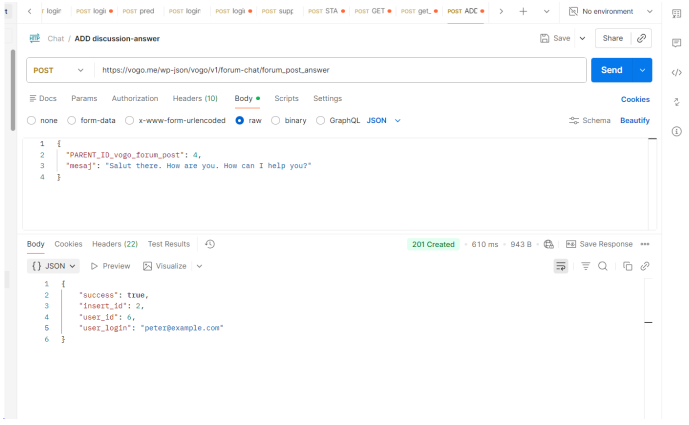
body:

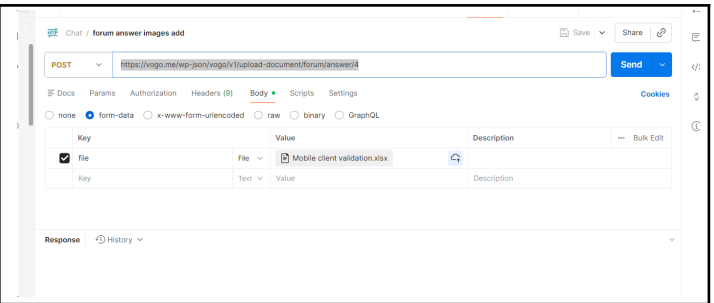
```
{
  "id": <thread id>
}
```

Bearer = for human operator = human operator bearer. For chatbot screen – client token



Add new message API:
https://vogo.me/wp-json/vogo/v1/forum-chat/forum_post_answer



Add new image to chat API: https://vogo.me/wp-json/vogo/v1/upload-document/forum/answer/4	
Expectation is like user from chatbot screen to discuss in chat with human operator. That's why we need to add a timer that refresh through ajax messages and display new messages – as in Hostinger chat.	

Here are some autogenerated specifications using reverse engineer on flutter mobile application that have already human operator chat implemented. May help you:

```
# Mobile Chat Reverse-Engineering Spec for Web

> Goal: help the web team implement chat behavior and UI as close as possible to the mobile app (`ChatListScreen` +
`ChatScreen`).

## 1) Screens and User Flows

### 1.1 Chat List (`ChatListScreen`)
- Displays user conversations with:
  - Peer avatar
  - Peer name
  - Last message preview (max 2 lines)
  - Relative timestamp (time / Yesterday / weekday / month+day)
  - Unread badge (`nr_to_read`) or read check icon
- Supports:
  - Pull-to-refresh
  - Infinite scroll pagination (load next page at list bottom)
  - Background refresh every **10 seconds** while tab is active
- Includes a **Compose** action to start a new support thread.

### 1.2 Compose New Thread
- First message flow:
  1. Fetch random support user (`get-random-user`)
  2. Send message via `start_or_update_chat_thread`
  3. Read `thread_id` from response
  4. Open chat detail on that thread.

### 1.3 Chat Detail (`ChatScreen`)
- Header shows:
  - Peer avatar
  - Peer name
  - Chat subtitle/title (`chatTitle`)
- Header actions (non-branded mode):
  - Video call
  - Voice call
- Timeline message types:
  - Text
  - Image
  - Video
  - File/document
  - Call events (voice/video with status)
- Bottom input:
  - Attachment button
  - Text input
  - Send button
- Sticky date indicator shown while scrolling.

---

## 2) Polling, Timers, Lifecycle

### 2.1 Timer Intervals
- Chat list polling: **10s**
- Chat detail polling: **30s** (`AppConstants.timerCharRefreshInterval`)

### 2.2 Start/Stop Rules
- Chat detail starts polling only when:
  - Current route is active (`didPush` / `didPopNext`)
  - App is in foreground (`resumed`)
- Chat detail stops polling on:
  - `didPushNext`
  - `didPop`
  - App `paused`
  - `dispose`
- Chat list stops polling when chat tab is inactive.
```

2.3 Web Parity Recommendation
- Keep same intervals for strict parity:
  - List: 10s
  - Detail: 30s
- Pause polling when browser tab/document is hidden.

---

3) API Contract (Used by Mobile)

3.1 Headers/Auth
- JSON requests:
  - `Content-Type: application/json`
  - `Authorization: Bearer <token>`
- File upload:
  - `multipart/form-data`
  - `Authorization: Bearer <token>`

3.2 Endpoints

A) Get chat discussions list
- `POST /forum-chat/get_forum_discussions_by_user?Content-Type=application/json`
- Request body:
```json
{
  "POST_TYPE": "CHAT",
  "page": 1,
  "limit": 10
}
```
- Response fields used:
```json
{
  "success": true,
  "userId": 123,
  "userName": "...",
  "current_page": 1,
  "per_page": 10,
  "total_pages": 5,
  "total_items": 43,
  "discussions": [
    {
      "id": 1001,
      "post_content": "last message",
      "last_answer_datetime": "2025-02-10 12:31:45",
      "nr_to_read": 2,
      "user_chat_id": 44,
      "username_chat": "Support",
      "user_chat_avatar": "https://...",
      "chat_show_title": "Order #123"
    }
  ]
}
```

B) Get thread detail metadata
- `POST /forum-chat/get_forum_discussion_by_id?Content-Type=application/json`
- Request body:
```json
{ "id": 1001 }
```

C) Get thread answers/messages
- `POST /forum-chat/get_answers_by_post_id?Content-Type=application/json`
- Request body:
```json
{ "post_id": 1001 }
```
- Response fields used in timeline:
```json
{
  "success": true,
  "answers": [
    {
      "ID": "555",
      "PARENT_ID_vogo_forum_post": "1001",
      "post_author": "44",
      "post_date": "2025-02-10 12:31:45",
      "mesaj": "Hello",
      "post_image": "https://.../file.jpg",
      "post_mime_type": "image/jpeg",
      "created_user": "44",
      "type": "Message | Voice call | Video call",
      "duration_seconds": "125",
      "is_video": "1",
      "missed_call": "0",
      "direction": "incoming|outgoing",
      "status": "CANCELLED|DECLINED|ENDED|RINGING"
    }
  ]
}
```

D) Send initial message / create thread
- `POST /forum-chat/start_or_update_chat_thread`
- Request body:
```json
{
  "chat_user_id": 44,
  "product_id": null,
  "order_id": null,
```

  "mesaj": "Hi"
}
```
- Response fields used:
```json
{
  "success": true,
  "thread_id": 1001
}
```

E) Send answer in existing thread
- `POST /forum-chat/forum_post_answer?Content-Type=application/json`
- Request body:
```json
{
  "PARENT_ID_vogo_forum_post": 1001,
  "mesaj": "Reply text"
}
```
- If `success == true`, mobile immediately refetches thread answers.

F) Upload chat file
- Two endpoint variants are present in code:
  1. `POST /chat-upload-document/forum/post/{threadId}`
  2. `POST /upload-document/forum/post/{threadId}`
- Multipart payload:
  - `file`: binary (`jpg/png/jpeg/gif/webp/mp4/mov/pdf`)
- Client-side limit: **50MB**
- On upload `200`, mobile refetches answers.

G) Get random support user
- `POST /support/get-random-user`
- Request body: `{}`
- Response field used: `supportUser.userId`

H) Call endpoints used by chat
- `POST /support/call`
- `POST /support/call/status`
- `POST /support/call/decline`
- `POST /support/call/end`
- `POST /support/call/cancel`

---

4) Message Mapping and State Logic

4.1 Message Type Detection
- If `answer.type` is `Voice call` or `Video call`:
  - Render as call card (`CustomMessage`-style block)
- Else:
  - `post_image` ends in `.mp4` / `.mov` -> Video message
  - `post_image` image extension -> Image message
  - `post_image` exists but not image/video -> File message
  - `mesaj` exists -> Text message

4.2 Optimistic UI + Reconciliation
- On local send:
  - Insert temporary pending message immediately
- On refresh/server response:
  - Match and replace pending items with confirmed server entries
  - Use dedup key pattern based on user + timestamp + text/image
  - Keep timeline sorted by `createdAt` descending (newest first in data source)

4.3 Call Card Status Mapping
- `CANCELLED`
  - Label: “Missed Voice/Video call”
  - Secondary text: “Tap to callback”
  - Icon color: red
- `DECLINED`
  - Label: “Voice/Video call”
  - Secondary text: “Declined”
  - Icon color: red
- `ENDED`
  - Label: “Voice/Video call”
  - Secondary text: duration (example: `2 min 5 sec`)
  - Icon color: green
- `RINGING`
  - Secondary text: “Ringing”

4.4 Delivery/Upload Indicators
- Current user text messages:
  - `pending=true` -> clock icon
  - otherwise -> check icon
- Image/video/file uploads:
  - `pending=true` -> “Uploading…”
  - `failed=true` -> “Failed to upload”

---

5) UI/UX Parity Guidance for Web

5.1 Chat List
- Compact row layout:
  - Circular avatar (approx. 44px)
  - Name + last message preview (2 lines max)
  - Trailing time and unread circular green badge
- Support pull-to-refresh behavior and infinite scroll.
- Floating **Compose** button with edit icon.

5.2 Chat Detail
- Bubble styles:
```

  - Current user: primary green background, white text
  - Peer: light gray `#F3F4F6`, dark text
- Bubble corner shape mirrors mobile asymmetric look.
- Text bubbles show:
  - Content
  - Time
  - Delivery indicator for current user
- Media rendering:
  - Image width around 200px
  - Inline video play/pause
  - File row with icon, filename, optional size, click-to-download/open
- Bottom input bar:
  - Attachment icon (left)
  - Text field
  - Send icon (right)
  - Subtle top shadow

5.3 Date Labeling Rules
- `Today`
- `Yesterday`
- Same week -> weekday name (`Monday`, etc.)
- Older -> `dd-MMMM-yyyy`
- Sticky date badge appears near top while scrolling.

---

6) Known Gaps / Validation Items

- There are two different upload endpoints in current mobile code; backend contract should be unified before web implementation.
- Data list is sorted descending by timestamp in mobile message source; web should explicitly define render order and virtualization strategy.
- Polling intervals differ (list 10s vs detail 30s); keep this if strict parity is required.
- `status` is used from answers payload for call cards; web should parse it directly from API payload.

---

7) Web Implementation Checklist (MVP Parity)

- [ ] Chat list with pagination and 10s polling
- [ ] Compose flow with thread creation (`start_or_update_chat_thread`)
- [ ] Chat detail with 30s polling + hidden-tab pause
- [ ] Optimistic text send
- [ ] File upload with 50MB client limit
- [ ] Message rendering for text/image/video/file/call
- [ ] Call card status mapping
- [ ] Sticky date + same date formatting logic
- [ ] Unread badge + read indicator in list
- [ ] Error states for send/upload/network timeout

